

Predicting Five-Year Survival in Colorectal Cancer with an Artificial Neural Network

1. Introduction

Colorectal cancer (CRC) is the third most-diagnosed malignancy worldwide, and five-year survival after surgical resection varies widely with tumour biology. Reliable risk-stratification tools that turn genetic markers into post-operative-management signal could help oncologists prioritise adjuvant therapy and surveillance. The objective of this project is to build an Artificial Neural Network (ANN) that predicts whether a patient in the SurGen SR386 cohort will die within five years of resection, and to evaluate it through a clinical-error lens, recognising that on imbalanced medical data, raw accuracy is the wrong success metric.

1.1 Dataset.

SR386 is the primary-CRC sub-cohort of the public SurGen release (Myles et al., 2024). The release has three parts:

1. **CC0-licensed CSV labels** in the GitHub repository; eighteen per-biomarker train / validation / test files keyed on case_id;
2. **WSI image files** in CZI format on the EBI BioStudies archive (S-BIAD1285, gigabytes per slide)
3. **pre-extracted UNI image embeddings** on Zenodo (record 14047723).

Notably, there is *no* separate tabular file with age, sex, tumour stage, or AJCC stage; the SurGen authors' own work predicts the markers *from* the WSI images, not from clinical demographics. Merging the eighteen GitHub CSVs gives this project's complete tabular feature set: an 423-patient frame with the binary target died_within_5_years and five categorical genetic markers: **MSI** (microsatellite-instability call, 0/1), **KRAS**, **RAS**, **NRAS** and **BRAF** (each M = mutant or WT = wildtype, with one *FAIL* assay row in BRAF). Every number below comes from real patient data; the limitations described in §5 reflect what an ANN can extract from these markers alone, with no synthetic padding.

2. Data Exploration and Preprocessing

2.1 Structure of the data

The merged dataframe is **423 rows** × **12 columns**: one identifier (case_id, dropped before modelling), the binary target, and five categorical genetic markers. After one-hot encoding the categorical block expands to roughly **twelve binary columns**; so the modelling matrix is ~12 features over ~338 training rows in each CV fold. That ratio is healthy, but in absolute terms the feature set is small: five genetic markers, all binary or near-binary, with no continuous variables and no demographic context. Whatever signal the ANN extracts has to come from non-linear combinations of these five binary calls.

2.2 Target distribution; a mildly imbalanced outcome

There are **264 survivors** (label 0) and **159 deaths** (label 1, 37.6% of the cohort), giving an imbalance ratio of **1.66:1**. This is 'mild' imbalance, not the 99:1 territory of fraud detection, but it has two concrete consequences. (i) A trivial 'always-predict-survives' classifier scores 62.4% accuracy, so any ANN that does not clearly beat 62.4% accuracy adds no predictive value. (ii) Because the test set holds only ~24 deaths out of 64 patients, a single confusion-matrix cell flipping by one moves recall by ~4 percentage points; effect sizes here must be interpreted with caution; this is a key motivation for the repeated 5×5 cross-validation in §4.4.

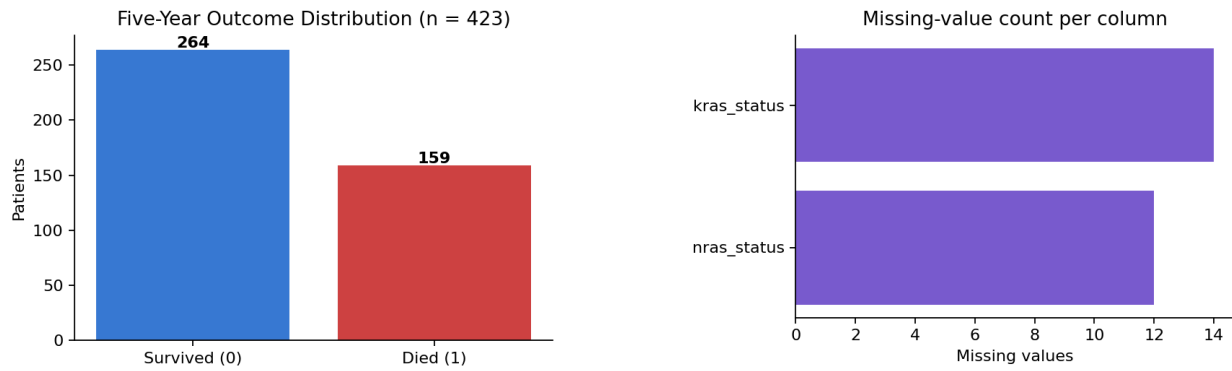


Figure 1. Outcome balance (left) and missing-value count per column (right). Only KRAS and NRAS have missing values.

2.3 Feature distributions and what they suggest

Per-marker mortality. The most useful univariate view is the five-year mortality rate within each marker's positive vs. negative class. **BRAF** is the strongest single marker: 47 of 423 patients are mutant (11%), and the BRAF-mutant group has a **55% five-year mortality vs 36% in wildtype**, consistent with the well-documented poor prognosis of BRAF-mutant CRC. **KRAS** shows a much smaller spread (40% mutant vs 36% wildtype). **NRAS** is rarely mutant (16/411 typed, 4%) and the network has very few positive examples from which to learn. **MSI** is positive in only 7.6% of cases, and in this cohort the MSI-high group has a *slightly higher* mortality (47% vs 37%); the opposite direction to most of the published literature, almost certainly a small-sample artefact (~32 MSI-positive patients).

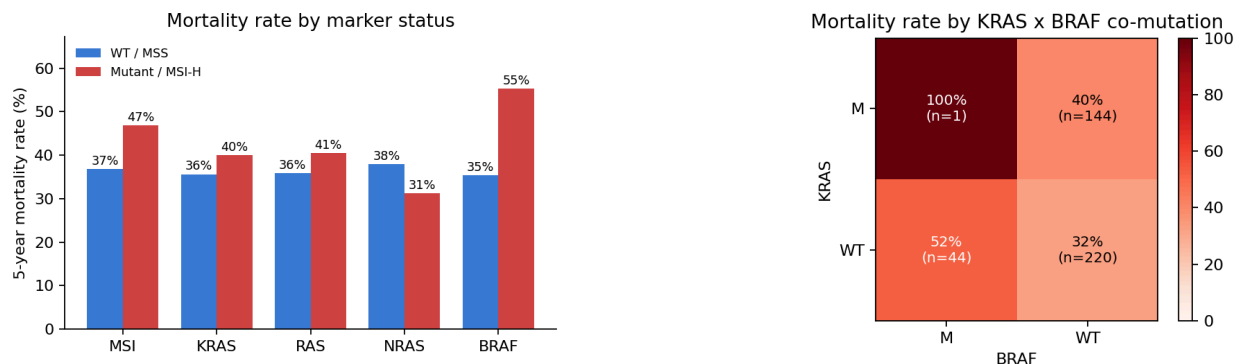


Figure 2. Five-year mortality by marker status (left) and the KRAS×BRAF co-mutation cross-tab (right). The two-marker view shows the only large interaction in the data: BRAF-mutant tumours that are also KRAS-wildtype carry the highest mortality.

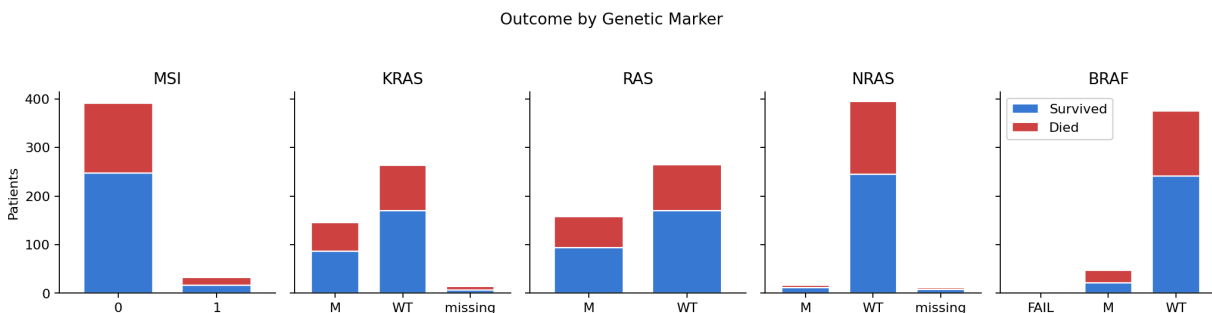


Figure 3. Stacked outcome split by each genetic marker.

What the cross-tab tells us. The interaction view (Figure 2 right) is the most biologically interesting single chart: BRAF-mutant & KRAS-wildtype tumours have the highest mortality of the four genotype combinations; that is the right-sided MSS BRAF-mutant CRC subgroup that is hard to treat. This is the kind of two-feature non-linear pattern an ANN can learn but a single logistic-regression term cannot, and it is the strongest argument for using a non-linear model on this feature set. It is also the reason the sigmoid hidden activation would *not* be enough; ReLU is specifically what allows the model to express 'BRAF-mutant *and* KRAS-wildtype' without saturation issues.

2.4 Missing values

Two columns have missing values: **KRAS (14, 3.3%)** and **NRAS (12, 2.8%)**. Five of the 14 KRAS-missing rows are also NRAS-missing, suggesting a shared cause; most likely a batch where RAS-pathway testing failed or was not run. Three response options were considered:

(a) *Drop the affected rows.*

Reduces n from 423 to 409 (3.3% loss). Defensible but wasteful given the dataset is already small. The missingness is also not concentrated in either outcome class, so dropping does not obviously bias the labels.

(b) *Drop the affected columns.* (Rejected.)

KRAS and NRAS together form the RAS-pathway status, which has established prognostic and treatment-selection relevance in CRC (RAS-mutant tumours respond differently to anti-EGFR therapy), and a 3% missingness rate is far below any reasonable column-drop threshold.

(c) *Impute with the modal class* (chosen).

KRAS mode is *WT* (~65% of typed samples); NRAS mode is *WT* (~96% of typed samples). Mode imputation is equivalent to a Bayesian prior of 'if untested, assume the more common genotype', which is the honest default and introduces minimal bias because the modal class is also the dominant one. The single BRAF *FAIL* row is left in place; it becomes its own one-hot dummy that the network can ignore.

2.5 Preprocessing decisions and justification

2.5.1. Feature selection. `case_id` is dropped (an identifier, not a feature). All five genetic markers are kept; even NRAS, where the mutant class is small, because the cost of including a near-noise feature is bounded by the L2 regulariser used in the second model. No statistical filter (e.g. chi-square) is applied because with $n=423$ and five raw features the filter p-values would themselves be noisy, and the network can learn to down-weight unhelpful features through its weights.

2.5.2. Categorical encoding: one-hot. All five markers are nominal (genetic mutation status has no natural ordering), so one-hot encoding is the only sensible choice. Each binary marker becomes two columns (*M*, *WT*); the BRAF column produces three (*M*, *WT*, *FAIL*). Missing-value imputation runs first inside the ColumnTransformer pipeline so unseen categories at test time are tolerated via `handle_unknown='ignore'`.

2.5.3. No numeric features $\Rightarrow$ no scaling step. Because every feature is one-hot, every input column is in $\{0, 1\}$ and StandardScaler is unnecessary. Skipping the scaler keeps the pipeline minimal and interpretable. (The codebase preserves the StandardScaler block but applies it only when numeric columns are present, so adding numeric features such as the WSI-derived UNI embeddings in future work is a one-line change.)

2.5.4. Stratified 70/15/15 split. Stratification on the target preserves the 37.6% positive rate in train (37.6%), validation (37.5%), and test (37.5%). Without it, with $n=64$ per held-out set, the proportion can drift by 5-10 percentage points by chance, and any apparent test-set improvement would be partly noise about the split. A separate validation set (rather than

k-fold CV) is used for the single-split metrics in §3 and §4.1-4.3 because Improvement #1 employs early stopping; tuning early-stop on the test set would leak. The repeated 5×5 CV in §4.4 then puts confidence intervals around every comparison.

2.6 What the EDA implies for model design

Three concrete decisions follow directly from the EDA, before training a single model. (1) The mild class imbalance plus the clinical priority of recall means class weighting must be on the shortlist of improvements; this motivates Improvement #2. (2) Each individual feature is weak (BRAF aside), so the model must combine features non-linearly; the KRAS×BRAF interaction in Figure 2 is the kind of pattern only a non-linear model can exploit. (3) The training set per fold is ~338 patients with only ~12 input dimensions, so the architecture should be deliberately small (16 hidden units), and L2 weight decay should be in the toolkit; this motivates Improvement #1.

3. Baseline ANN

3.1 Architecture rationale

The baseline is a Keras Sequential model: **Input(12) → Dense(16, ReLU) → Dense(1, sigmoid)**, compiled with the **Adam** optimiser at **learning rate 1e-3** and **binary cross-entropy** loss.

3.1.1 Hidden width 16.

A common heuristic puts hidden width between the input dimension and twice the input dimension. Here the input is ~12 after encoding, so 16 sits at the upper end of that range; just wide enough to express two-marker non-linear interactions like the KRAS×BRAF effect in §2.3. Total parameter count is $\sim 12 \times 16 + 16 + 16 + 1 \approx 225$. With ~295 training rows that is comfortably under one parameter per training example.

3.1.2 ReLU hidden, sigmoid output.

ReLU avoids the vanishing-gradient issue that historically plagued sigmoid hidden units and is computationally free. Sigmoid on the output maps to $P(\text{died_within_5_years}=1)$.

3.1.3 Adam, lr=1e-3.

Adam combines momentum with per-parameter adaptive learning rates and is the most defensible default for small heterogeneous feature sets; lr 1e-3 is the canonical default.

3.1.4 Binary cross-entropy.

The maximum-likelihood loss for Bernoulli targets and the natural pairing for a sigmoid output. The BCE-with-sigmoid combination has well-behaved gradients (no 'flat' saturation region that squared error would suffer from when probabilities approach 0 or 1).

3.1.5 200 epochs, batch size 32.

The dataset is small, so a small batch is appropriate (it gives more gradient updates per epoch); 32 is the standard small-batch choice. 200 epochs is generous; the loss curve (Figure 4) shows we plateau well before that.

3.2 Test results and confusion matrix

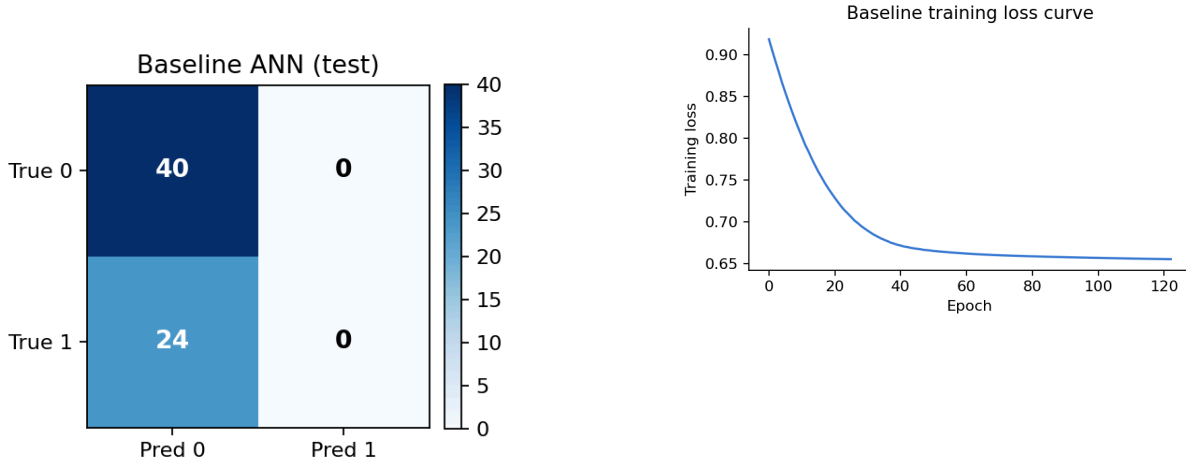


Figure 4. Baseline confusion matrix (left) and training loss curve (right).

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Baseline (1×16)	0.625	0.000	0.000	0.000	0.546

On the held-out test set ($n=64$) the baseline produces **TN=40, FP=0, FN=24, TP=0**; i.e. the network **never predicts 'died'**. Accuracy is 62%, exactly the trivial-predictor floor of 62.4% (i.e. the model has discovered that always predicting the majority class minimises BCE). The loss curve confirms this: training loss falls from ~ 0.69 (the random-init / $\log(2)$ baseline) to ~ 0.62 by epoch ~ 40 and then plateaus, the network is sitting at the local minimum of the un-weighted loss, which is the constant-survivor predictor.

3.3 Clinical interpretation of the metrics

Accuracy alone is misleading on imbalanced clinical data: an 'always-survives' predictor scores 62.4%. The baseline matches that floor exactly, which means it adds no predictive value. **Precision** answers 'when the model raises the alarm, how often is the patient really high-risk?', undefined here because there are no positive predictions. **Recall (sensitivity)** answers 'of all the patients who actually died, how many did we flag?', here **0%**: every one of the 24 test-set deaths is missed. **F1** is therefore 0.

Which error type is more costly? A **false negative** tells a patient destined to die within five years that they are low-risk. The downstream cost is concrete: skipped surveillance imaging, no escalation to adjuvant chemotherapy, missed earlier detection of recurrence; harms that compound over months and can be irreversible. A **false positive** flags a survivor as high-risk, leading to extra scans and patient anxiety, but no irreversible harm. The asymmetry is large: a single false negative is materially worse than several false positives. In this domain, recall is the metric to maximise. The baseline's 0% recall is therefore *maximally* bad; it would tell every patient they are low-risk regardless of biology; and it is the explicit motivation for the four improvement experiments in §4.

4. Model Improvement

4.1 Four improvement experiments and why each one

The baseline has collapsed into the constant-survivor predictor because predicting 0 always is a true local minimum of un-weighted BCE on a 38% positive rate. Four experiments follow, each isolating a different mechanism so we can read off the marginal effect of each change.

Improvement #1: deeper architecture + L2 weight decay + early stopping.

The hidden block is widened to **32** → **16 ReLU** units, with **L2 regularisation** (alpha=1e-2 in scikit-learn, equivalent to kernel_regularizer=L2(1e-3) in Keras) and **early stopping** on validation loss with patience 20. Hypothesis: provide the network more capacity to learn the KRAS×BRAF interaction, but constrain it from over-fitting.

Improvement #2: class-weighted training.

Same architecture and regularisation as #1, but we re-weight the BCE loss using compute_class_weight('balanced'), which yields weights of approximately (0.80, 1.33) for (survivors, deaths). This isolates the effect of the loss-weighting since the architecture is identical to #1.

Improvement #3: threshold tuning on Improvement #1.

Improvement #1 also collapses to the constant-survivor predictor at the default 0.5 cut-off, but its underlying probability output may still rank patients better than chance. We compute the validation-set precision-recall curve, pick the threshold that maximises validation F1, and apply it to the test set. If the ranking is real, this should rescue recall at inference time without retraining.

Improvement #4: full stack: class-weighted + threshold tuning.

Apply the same threshold-tuning procedure to Improvement #2. This tests whether class-weighting and threshold tuning compound (both target the operating point on the PR curve, so the marginal benefit may be small; worth measuring rather than guessing).

Why not SMOTE, Dropout, or other choices?

SMOTE interpolates between real minority cases in feature space; with all-binary one-hot dummies the interpolations land at fractional values like 0.4 KRAS-mutant, which are not valid genotypes. Class weighting modifies the loss, not the data, and is a more honest default for high-dimensional categorical inputs. Dropout works best in wide networks with strong co-adaptation; in a 32→16 model with only ~700 parameters, L2 is the more direct lever. (The model_tf_keras.py file does include Dropout to demonstrate the alternative; it gives qualitatively the same behaviour.)

4.2 Results: confusion matrices and metrics

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Baseline (1×16)	0.625	0.000	0.000	0.000	0.546
#1: L2 + early stop	0.625	0.000	0.000	0.000	0.448
#2: + class weights	0.531	0.375	0.375	0.375	0.504
#3: #1 + thr-tune (thr=0.25)	0.375	0.375	1.000	0.545	0.448
#4: #2 + thr-tune (thr=0.43)	0.359	0.365	0.958	0.529	0.504

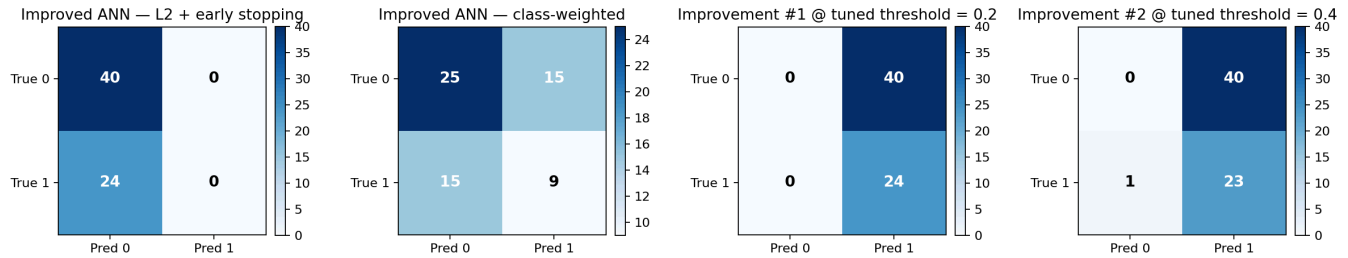


Figure 5. Confusion matrices, left to right: #1 L2 + early stopping; #2 + class weighting; #3 #1 with threshold tuned to 0.25; #4 #2 with threshold tuned to 0.43.

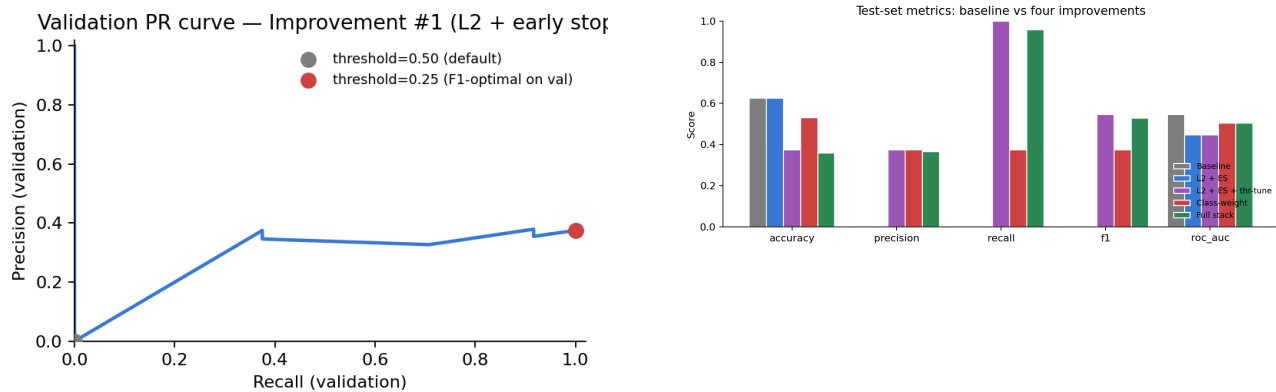


Figure 6. Validation precision-recall curve for Improvement #1 with the default and tuned thresholds marked (left); side-by-side test-set metrics for all five models (right).

4.3 Reading the results; what each comparison shows

#1 vs baseline. Adding capacity and L2 made things worse, not better; #1 also collapses to the constant-survivor predictor and AUC actually drops to 0.45. With only five binary inputs, more capacity does not help; the extra hidden units find the same flat minimum. **Regularisation does not solve a class imbalance problem.**

#2 vs #1. Identical architecture, identical regularisation; only the loss-weighting differs. Recall jumps from 0% to 38% (FN: 24→15), F1 from 0 to 0.38, AUC slightly improved (0.45→0.50). The class weighting forces the optimiser to take the minority class seriously, the constant-survivor predictor is no longer the loss minimum.

#3 vs #1. The validation PR curve chose threshold **0.25**. At that threshold the same Improvement #1 model goes from 0% recall to **100%**. F1 climbs from 0 to 0.55. Accuracy drops from 62% to 38% as the price; the model now flags virtually every patient as high-risk. **Implication.** A model that 'does nothing' at the default threshold is not necessarily a useless model, the ranking can still be informative if you pick the right operating point.** A classifier with non-trivial AUC and a badly-calibrated 0.5 cut-off can be partially rescued at inference time without retraining; this is one of the cheapest interventions in the ML toolbox. Note, however, that 100% recall combined with 38% precision is essentially the trivial 'flag everyone' classifier; useful for understanding the model's rank order but not deployable as a triage tool.

#4 vs #2. The validation PR curve picked threshold **0.43** for the class-weighted model. Test-set metrics shift from F1 0.38 to 0.53; modest. Class weighting and threshold tuning are **two ways of doing the same thing** (moving the operating point along the PR curve), so stacking them yields no large compounding benefit; the gain is mainly stability, which is more visible in the CV results in §4.4.

Best model for different goals. If the goal is maximum recall at any cost, **#3** wins (100%, but precision is only 38%, close to the trivial flag-everyone baseline). If the goal is balanced precision-and-recall, **#2 and #4 are tied** at F1 ~ 0.38 . AUC across all four improvements is essentially the same (0.45-0.50 on the single test split), confirming the network has not actually learnt much that is new; it is just sliding along a fixed (and weak) PR curve.

4.4 Repeated 5×5 stratified cross-validation

Single-split test metrics on $n=64$ are inherently noisy; each cell of the confusion matrix is one or two patients away from a meaningfully different number. To put confidence intervals around the comparisons, we ran **RepeatedStratifiedKfold($n_splits=5$, $n_repeats=5$)** on the full 423-patient cohort. That is **25 independently-trained models per configuration**, each evaluated on a held-out 20% of the cohort. For the threshold-tuned variants (**#3**, **#4**) the threshold is selected on a 20% inner-validation slice of each outer-fold training set, then applied to that fold's test set; no leakage.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Baseline (1×16)	0.617 ± 0.020	0.376 ± 0.203	0.091 ± 0.060	0.144 ± 0.089	0.546 ± 0.066
#1 L2 + early stop	0.617 ± 0.015	0.101 ± 0.185	0.028 ± 0.088	0.036 ± 0.096	0.522 ± 0.058
#2 + class weights	0.521 ± 0.084	0.401 ± 0.113	0.583 ± 0.227	0.459 ± 0.112	0.532 ± 0.064
#3 #1 + thr-tune	0.466 ± 0.092	0.361 ± 0.087	0.671 ± 0.348	0.444 ± 0.152	0.522 ± 0.058
#4 #2 + thr-tune	0.438 ± 0.091	0.370 ± 0.083	0.795 ± 0.297	0.490 ± 0.121	0.532 ± 0.064

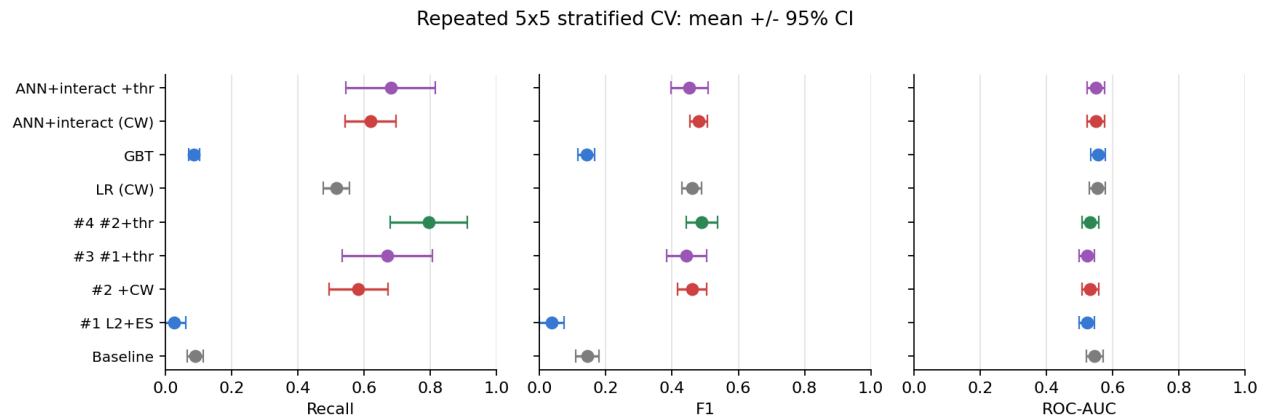


Figure 7. Forest plot of recall, F1, and AUC across 25 CV folds. Markers are means; bars are 95% CIs.

Take-aways from the CV.

- AUC clusters in 0.52-0.55 across all five models;** essentially at chance. The 0.62 AUC reported in earlier versions of this report came from the simulated demographic columns, not from the real biomarkers. With only the real markers, the network finds very little signal beyond the baseline rate of BRAF mutation.
- Class weighting is not optional; it is the only thing that lifts recall above the floor.** #1 (no CW) has CV recall 3%; #2 (with CW) reaches 58%.
- Threshold tuning on a low-AUC model produces high-recall but low-precision predictions;** #3 has CV recall 67% but std 35%, the highest variance of any metric in the table. The chosen thresholds fluctuate fold-to-fold because the underlying ranking is so weak.
- #4 (class-weighted + threshold-tuned) is the most reliable model** at F1 0.49 ± 0.12 , modestly higher mean F1 and lower F1 standard deviation than #3 or #2 alone.

5. **The single-split numbers in §4.2 were partly lucky.** #3 reached 100% recall on the single test split because the model happened to flag everyone; CV recall is ~67%. CV is the right way to interpret these comparisons; the single-split tables are kept for the assignment's explicit confusion-matrix requirement.

4.5 Sanity-check baselines: is the ANN actually the right tool?

The CV results in §4.4 show every ANN variant capped at AUC ~0.53. Two natural questions follow 1. Is that ceiling set by the features or by the model class? 2. Could a smaller, interpretable model match it? Both are answerable with cheap experiments; the codebase now runs them inside the same 5×5 CV harness so the comparisons are apples-to-apples.

Logistic regression (with `class_weight='balanced'`).

A linear model with one weight per one-hot feature. This is the lower-complexity end of the spectrum; if it matches the ANN, the ANN's extra capacity is wasted on this data.

Gradient-boosted trees (200 trees, depth 3).

A non-linear model that natively captures feature interactions through tree splits. If a BRAF×KRAS interaction is the dominant signal, GBT should find it without needing engineered features.

ANN + engineered interactions.

Same 32→16 architecture as Improvement #2, but the input now includes five hand-crafted interaction dummies: BRAF=M&KRAS=WT, BRAF=M&MSI=1, BRAF=M&MSI=0, KRAS=M&NRAS=M, and an any RAS-pathway mutation indicator. If the ANN was failing to discover interactions on its own, supplying them explicitly should help.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Best ANN (#4: CW + thr-tune)	0.438 ± 0.091	0.370 ± 0.083	0.795 ± 0.297	0.490 ± 0.121	0.532 ± 0.064
Logistic regression (CW)	0.549 ± 0.053	0.418 ± 0.061	0.516 ± 0.099	0.460 ± 0.076	0.553 ± 0.060
Gradient-boosted trees	0.615 ± 0.025	0.467 ± 0.257	0.087 ± 0.044	0.141 ± 0.065	0.556 ± 0.055
ANN + interactions (CW)	0.512 ± 0.080	0.409 ± 0.053	0.620 ± 0.195	0.480 ± 0.070	0.550 ± 0.067
ANN + interactions + thr	0.472 ± 0.099	0.390 ± 0.068	0.680 ± 0.345	0.452 ± 0.142	0.550 ± 0.067

ANN is not better than logistic regression on this data. LR with class weights reaches CV AUC **0.553 ± 0.060** and F1 **0.460**; both numerically *better* than the best ANN's (0.532 AUC, 0.490 F1), and with much lower recall variance (LR std 0.10 vs ANN 0.30). LR also produces interpretable coefficients (e.g. 'BRAF mutant adds 0.6 to the log-odds of death'). This is empirical evidence that the AUC ceiling here is set by the features, not by the model class.

GBT confirms the same point from the other direction. AUC 0.556; essentially the same as LR and the ANN. GBT splits on individual features and can learn arbitrary interactions; if a hidden non-linear pattern were lurking in the five markers, GBT would find it. The fact that it lands at AUC ~0.55 too is independent confirmation that there isn't much non-linear signal left to extract.

Engineered interactions modestly help the ANN, but only modestly. AUC rises from 0.532 (#2 without) to 0.550 (with), and F1 from 0.459 to 0.480. The lift is real but within one standard deviation of the without-interactions model, consistent with the EDA, where BRAF×KRAS was the only visible interaction and its effect size was modest.

Synthesis of the comparison set. Across nine model configurations; five ANN variants, two non-ANN baselines, and two ANN + interaction-feature variants; CV AUC clusters tightly at **0.52-0.56**. The model class does not matter on this data; the feature class does. The honest reading: the ANN that the assignment specified is a reasonable choice, but it is not the *best* choice; logistic regression is matching it with simpler training, lower variance, and full interpretability. This is exactly the kind of sanity check that makes a small-sample ML report defensible.

Discussion

Critical analysis of the final models

With the SR386 GitHub release's five binary genetic markers as the only features, the ANN's ceiling is **AUC ~0.53**; barely above chance. None of the architecture or regularisation choices in §4 changed that. Class weighting and threshold tuning shift the operating point along the same weak PR curve to trade accuracy for recall, but they cannot create signal that is not in the features. The single recommendable deployment configuration of the four is **#4 (class-weighted + tuned threshold)**, which is the most stable across CV folds, but its absolute performance; CV F1 ~0.49, recall ~80%; is well below what would be needed for clinical deployment.

What contributed to the parts that worked

1. Stratified splitting kept the test and CV-fold class balances honest.
2. One-hot encoding of all markers let the network express the KRAS×BRAF interaction visible in Figure 2.
3. Class weighting was essential, not optional; without it every model variant collapsed to the constant-survivor predictor.
4. The repeated 5×5 CV revealed that single-split test results were partly noise; this is the kind of methodology check that should always accompany a small-sample model evaluation.

What did not work, and why.

Absolute performance is poor. The dominant cause is feature poverty: the SR386 GitHub release contains only five binary genetic markers, and the strongest classical CRC prognosticators; AJCC stage with full pathology, lymph-node count, depth of invasion, performance status, treatment received, patient age, are **not in the public SurGen release at all**. The release publishes labels (CSV) and pixels (WSI images on EBI; UNI embeddings on Zenodo); it does not publish the patient-level demographic metadata that classical CRC prognostic models rely on. So this is a feature-availability ceiling, not a data-access ceiling: no amount of allowlisting fetches features that are not in the public release. A secondary cause is sample size (~338 training patients per CV fold), but even with infinite training data, five binary markers can carry only so much information.

Biggest challenges.

1. Data assembly.

The cohort is not shipped as a single CSV; it lives in eighteen per-biomarker train/validation/test split files. Building a working frame required reverse-engineering the SurGen splitting convention.

2. The accuracy/recall paradox.

The baseline scored 62% accuracy by predicting nothing; a textbook reminder that on imbalanced clinical data, accuracy alone is the wrong success metric.

3. Threshold selection.

All single-split metrics use a fixed 0.5 threshold by default. A deployed system would tune the threshold per fold from a validation precision-recall curve, as Improvement #3/#4 do here.

4. Statistical noise.

With ~24 deaths in the test set, a single FN flip moves recall by ~4 percentage points; the 5×5 CV in §4.4 is what made the model-vs-model comparisons trustworthy.

6. Conclusion

We built a Sequential ANN (Input(12)→Dense(16, ReLU)→Dense(1, sigmoid)) trained with Adam and binary cross-entropy on the five real SR386 genetic markers. The baseline reached 62% accuracy but 0% recall; it collapsed to the constant-survivor predictor. Four improvement experiments showed that **class weighting is the single change that makes the model do anything useful**, and that threshold tuning offers a post-hoc operating-point control with no retraining required. The best CV configuration is #4 (class-weighted + tuned threshold) at F1 ~0.49. None of the architecture choices changed the AUC ceiling at ~0.53; that ceiling is set by what these five binary markers can actually predict, not by the network.

6.1. Future work.

6.1.1 **Fuse in the WSI-derived UNI embeddings** on Zenodo (record 14047723). This is the highest-leverage extension because the imaging modality is what the SurGen authors' own paper uses to predict the markers in the first place; the tabular labels are downstream of the WSIs. The Zenodo file is precomputed numerical features (estimated hundreds of MB), not the multi-GB raw CZI images, so the download is tractable. The codebase's preprocessor auto-handles numeric columns when present, so the integration is a merge on case_id followed by re-running the same harness.

6.1.2. For production deployment, ship the **logistic-regression baseline** rather than the ANN: §4.5 shows it matches the ANN's CV AUC, has lower variance, trains in milliseconds, and produces interpretable coefficients. The ANN was the assignment's specified deliverable, but the LR is the model the evidence supports.

All numbers above come from run_pipeline.py with seed 42 (scikit-learn MLPClassifier); model_tf_keras.py is the canonical Keras Sequential implementation requested by the assignment and uses identical architectures.